

Deep Learning Notes

Simple Concepts • Clear Explanations • Interview Preparation • GitHub Reference

Core mental model: Input → Weighted Sum + Bias → Activation → Prediction → Loss → Backpropagation → Gradient → Optimizer → Updated Weights → Repeat.

1. What is Deep Learning?

Deep Learning is a type of Machine Learning that uses neural networks with multiple layers to learn complex patterns from data.

Key points: learns features automatically; works with large amounts of data; used in images, text, audio, and time series.

Examples: image recognition, house-price prediction, language understanding.

2. Neuron & Neural Network

A neuron takes inputs, multiplies them by weights, adds a bias, and applies an activation function.

Weighted sum: $z = \sum(x_i \times w_i) + b$

Activation: $a = f(z)$

Remember: Inputs → Weighted Sum + Bias → Activation → Output.

3. Neural Network Layers

Input Layer: receives features.

Hidden Layer(s): learns patterns and relationships; layer/neuron count is generally selected through experimentation and validation.

Output Layer: produces the prediction. Binary classification usually uses 1 output neuron; multiclass classification uses N outputs; regression usually uses 1 output.

4. Activation Functions

Activation functions introduce non-linearity, helping networks learn complex patterns.

Common: Sigmoid, ReLU, Leaky ReLU, Softmax, Tanh, ELU, GELU.

5. Seven Common Activation Functions

Function	Range	Use	Drawback
Sigmoid	(0,1)	Binary classification; 1-neuron output	Vanishing gradients; not zero-centered
ReLU	[0,∞)	Hidden layers; CNNs; general DL	Dying ReLU
Leaky ReLU	(-∞,∞)	Hidden layers; reduce dying ReLU	Slope must be chosen
Softmax	0–1 each; sum=1	Multiclass output	Costly with very large class counts
Tanh	(-1,1)	Hidden layers; some sequence models	Vanishing gradients; saturation
ELU	(-α,∞)	Hidden layers / DNNs	More computation than ReLU
GELU	≈(-0.17,∞)	Transformers / modern DL	More complex/computationally expensive

Sigmoid

Formula: $f(x) = 1 / (1 + e^{-x})$

Uses: Binary classification; output layer with 1 neuron.

Drawback: Vanishing gradient; not zero-centered.

ReLU

Formula: $f(x) = \max(0, x)$

Uses: Hidden layers; computer vision; CNNs.

Drawback: Dying ReLU.

Leaky ReLU

Formula: x if $x \geq 0$; αx if $x < 0$

Uses: Hidden layers; reduce dying-ReLU problem.

Drawback: Negative slope must be chosen.

Softmax

Formula: $f(x_i) = e^{x_i} / \sum e^{x_j}$

Uses: Multiclass output; N neurons for N classes.

Drawback: Can be expensive for very large class counts.

Tanh

Formula: $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

Uses: Hidden layers; some DL/RNN use.

Drawback: Vanishing gradients; saturation.

ELU

Formula: x if $x \geq 0$; $\alpha(e^x - 1)$ if $x < 0$

Uses: Hidden layers / DNNs.

Drawback: More computationally expensive.

GELU

Formula: $\approx 0.5x[1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3))]$

Uses: Transformers and modern NLP/DL.

Drawback: More complex and costly.

6. Batch Normalization

Batch Normalization normalizes activations within a mini-batch to make training more stable.

$x_{\text{norm}} = (x - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$; then $y = \gamma x_{\text{norm}} + \beta$.

Benefits: can stabilize gradients, make convergence faster/more stable, help with vanishing/exploding-gradient issues, and provide a regularizing effect.

Flow: Input → Layer → Batch Normalization → Activation → Output.

Used in: CNNs, neural networks, some RNNs, Transformers, and other DL models.

7. Optimization Methods

Gradient Descent: $w = w - \alpha(\partial J / \partial w)$. It updates parameters toward lower loss.

Batch GD: entire dataset per update; stable but potentially slow.

Mini-Batch GD: small batch per update; efficient and common in DL.

SGD: one sample per update; frequent but noisy updates.

Momentum: uses previous-gradient information for smoother/faster movement.

AdaGrad: adapts learning rates; useful for sparse features, but rates can become very small.

RMSProp: uses an exponentially weighted average of squared gradients.

Adam: combines momentum-like first-moment behavior with adaptive second-moment scaling; widely used and often converges quickly.

8. Local vs Global Minimum

Local minimum: lower than nearby points but not necessarily the lowest overall.

Global minimum: lowest loss across the entire landscape.

Deep-learning loss landscapes can be high-dimensional and non-convex, with flat regions and saddle points.

Improve optimization with: better initialization, Momentum/Adam, learning-rate scheduling, and regularization.

9. Complete Training Flow

Input → Weighted Sum + Bias → Activation → Prediction → Loss → Backpropagation → Gradients → Optimizer → Weight Update → Better Prediction → Repeat.

10. Epoch, Batch Size & Iteration

Epoch: one complete pass through the training dataset.

Batch: a portion of the dataset processed together.

Iteration: one parameter-update step for one batch.

Example: 1,000 records with batch size 100 → about 10 iterations per epoch.

11. ANN vs Traditional ML

Traditional ML: often needs feature engineering; excellent for many tabular problems; examples include Random Forest and XGBoost.

Neural Networks: can learn useful representations automatically and are powerful for images, text, audio, and other complex data; examples include ANN, CNN, RNN and Transformers.

Important: neural networks are not always better. Model choice depends on the problem, data, resources and evaluation.

12. ANN — Artificial Neural Network

ANN generally means a standard feed-forward neural network.

Common uses: classification, regression, and prediction.

Example: Customer information → ANN → Will customer leave?

13. CNN — Convolutional Neural Network

CNNs are especially useful for images and computer vision.

Typical flow: Image → Convolution → Feature extraction → Pooling → More layers → Classification.

Applications: face recognition, object detection, medical image analysis, number-plate recognition, and image classification.

14. RNN — Recurrent Neural Network

RNNs were designed for sequential data such as text, speech, and time series. Traditional RNNs have limitations with long-term dependencies, leading to architectures such as LSTM and GRU.

15. LSTM — Long Short-Term Memory

LSTM is a recurrent architecture designed to better retain useful information over longer sequences. It has been used for time-series forecasting, text generation, NLP, and speech-related tasks.

16. Real-Life Examples

House Price: area + bedrooms + location + age + distance → neural network → price.

Image Recognition: image → learned visual patterns such as edges, shapes and textures → classification. CNNs are especially important for computer vision.

Prescription Analysis: prescription image → image processing/OCR → text extraction → information extraction → medicine/dosage/patient information; NLP/LLM processing can organize the extracted information.

17. Parameters vs Hyperparameters

Parameters: learned by the model, such as weights and biases.

Hyperparameters: selected/configured by us, such as learning rate, epochs, batch size, hidden layers, and number of neurons.

Memory trick: Parameters are learned; hyperparameters are set by us.

18. Overfitting

Overfitting occurs when a model performs very well on training data but poorly on unseen data.

Example: Training accuracy 99% and validation accuracy 75% may indicate overfitting.

Common techniques: Dropout, regularization, early stopping, data augmentation, and more training data.

19. Important Interview Terms

Neuron • Input • Weight • Bias • Activation • Forward Propagation • Loss • Backpropagation • Gradient • Gradient Descent • Optimizer • Learning Rate • Epoch • Batch Size • Iteration • Parameter • Hyperparameter • Overfitting • Underfitting

20. Interview Answer — What is a Neural Network?

"A neural network is a machine learning model inspired by the way biological neurons process information. It consists of input, hidden, and output layers. Each neuron applies weights and bias to its inputs and passes the result through an activation function. During training, the model calculates loss, uses backpropagation to obtain gradients, and an optimizer updates the weights to reduce loss. Repeating this over epochs allows the network to learn patterns and make predictions on new data."

21. Final Story to Remember

Training Data → Initial Weights → Forward Propagation → Prediction → Loss → Backpropagation → Gradient Descent/Optimizer → Updated Weights → Better Prediction → Repeat.

One sentence: A neural network learns by adjusting its weights and biases so that its predictions become closer to the actual answers.

GitHub Note

Suggested repository structure: README.md • notes/Deep_Learning_Notes.pdf • notebooks/ • src/ • models/ (when appropriate). For large model files, consider Git LFS or release storage.